CHAPTER 1

Using and Creating APIs

As a software engineer, you are likely familiar with building applications that interact with various external services and data sources. One of the most common methods for communication and integration is through HTTP APIs (Application Programming Interfaces). HTTP APIs provide a standardized way for applications to exchange data and functionality over the internet.

This chapter will introduce you to the world of HTTP APIs and explore how you can leverage them in your software development projects. We will cover the fundamental concepts, techniques, and best practices for effectively working with HTTP APIs.

An HTTP API allows two software systems to communicate and exchange information using the Hypertext Transfer Protocol (HTTP). It enables your application to make requests to an API server and receive responses in a structured format, such as JSON (JavaScript Object Notation) or XML (eXtensible Markup Language).

Using HTTP APIs offers a range of benefits for software engineers. It allows you to leverage external services and data sources, enabling your application to access functionality or retrieve valuable information from third-party systems. This opens up opportunities for integration with popular platforms, social media networks, payment gateways, geolocation services, and much more.

Throughout this chapter, we will explore various aspects of working with HTTP APIs, including:

- API endpoints and methods: Understanding how to interact with an API involves identifying the available endpoints and the supported methods, such as GET, POST, PUT, DELETE, and so on. We will discuss how to construct API requests and handle different response formats.
- Authentication and authorization: Many APIs require authentication to ensure secure access and protect sensitive data. We will delve into different authentication mechanisms, including API keys, tokens, OAuth, and other authentication protocols commonly used in API integrations.
- Request parameters and payloads: APIs often accept additional parameters or payloads to customize the request or send data for processing. We will explore how to pass query parameters, request headers, and request bodies when interacting with APIs.
- Error handling and status codes: Learning how to handle errors and interpret status

codes returned by APIs is crucial for building robust and resilient applications. We will discuss common status codes and best practices for handling various scenarios gracefully.

- Rate limiting and throttling: Many APIs impose restrictions on the number of requests you can make within a given timeframe to prevent abuse and ensure fair usage. We will cover techniques for handling rate limiting and implementing efficient strategies to manage API quotas.
- API documentation and testing: Proper documentation is essential for understanding an API's capabilities, endpoints, and expected behavior. We will explore how to read and interpret API documentation, as well as techniques for testing and validating API integrations.

By mastering the art of using HTTP APIs, you will expand your development toolkit and gain the ability to seamlessly integrate your applications with external services, leverage their functionalities, and build powerful, interconnected systems.

So, let's dive into the world of HTTP APIs and uncover the endless possibilities they offer for enhancing your software engineering projects.

1.1 Setting up an API

Create a file called `server.js`. This will be a basic webserver that we will use to test our an API. This is called a *RESTful API*. Write the following code to create a basic server for our API.

```
const express = require("express");

const app = express();
const PORT = 3000;

const student = {
  student_id: 42,
  name: "John Smith",
  major: "Computer Science",
  credits_completed: 88
};

app.get("/students/42", (req, res) => {
  res.json(student);
});

app.listen(PORT, () => {
  console.log(`API running at http://localhost:${PORT}`);
});
```

Then, run the following

```
npm init -y
npm install express
node server.js
```

- Endpoint
- Review HTTP methods
- headers
- Body

1.2 Requests with cURL

Curl, sometimes stylized as `cURL` or `curl`.

The simplest curl command

```
curl http://example.com
```

which will return the `html` source code of the website http://example.com.

```
<!doctype html>
<html lang="en">

<head>
    <title>Example Domain</title>
    <link rel="icon" href="data:,">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
        body {
            background: #eee;
            width: 60vw;
            margin: 15vh auto;
            font-family: system-ui, sans-serif
        }

        h1 {
            font-size: 1.5em
        }

        div {
            opacity: 0.8
        }
```

```
23
24          a:link,
25          a:visited {
26              color: #348
27          }
28      </style>
29  </head>
30
31  <body>
32      <div>
33          <h1>Example Domain</h1>
34          <p>This domain is for use in documentation examples without needing
            ↪  permission. Avoid use in operations.</p>
35          <p><a href="https://iana.org/domains/example">Learn more</a></p>
36      </div>
37  </body>
38
39  </html>
```

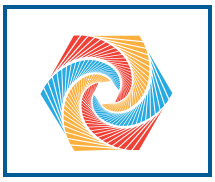
That's all great, but we can put cURL to work on much more laborious tasks. Going back to our created api example:

FIXME talk about API keys, API requests, how it interacts with HTTP requests, why keys should not be shared

---

Appendix A

Answers to Exercises



INDEX